Characterizing SIMD and Multithreaded GEMM Performance on the Hexagon DSP

Anonymous Author(s)
Anonymous Institution
Anonymous Email

—The DSPs found in mobile and embedded systems-on-chip increasingly carry wide vector units, which makes them attractive low-power targets for dense compute kernels. Yet three practical questions are seldom answered together, and almost never on real hardware: how far the vector unit alone carries a kernel, how much the DSP's hardware multithreading adds on top, and how a hand-written kernel compares to the optimised library shipped in the vendor SDK. We answer them for dense single-precision matrix multiplication (GEMM) on the Hexagon DSP of a production Qualcomm QCS6490, using six kernel variants that span scalar and vector execution under two thread-scheduling policies, all invoked from the CPU over FastRPC and measured with the on-chip performance counters against the vendor library qhblas. A compact hand-written vector kernel reaches the library's throughput at scale, which shows that the SIMD mapping and memory layout, rather than proprietary tuning, account for most of its performance. The vendor GEMM turns out to be single-threaded, so layering QuRT worker threads on the same kernel surpasses it by up to $2.6\times$. Vector multithreading saturates exactly at the number of HVX contexts the part provides, two on this SoC, and dynamic block scheduling avoids the load-imbalance cliffs that static assignment suffers. We also quantify a consequence of our own block-to-vector mapping, not a hardware quirk: a tile edge narrower than the vector width leaves zero full vectors per block, degrading the kernel to scalar execution at a cost of two orders of magnitude. Finally, an end-to-end comparison against the host CPU shows that offloading pays only when the kernel is vectorised and the problem is large enough to amortise the FastRPC round trip; offloading the scalar loop is a net loss.

***Index Terms*—DSP, SIMD, vector unit, GEMM, multithreading, vendor library, Hexagon, HVX, QuRT, FastRPC.**

I. Introduction

Heterogeneous mobile systems-on-chip pair a general-purpose CPU with specialized accelerators, and among these the digital signal processor (DSP) is the workhorse for signal-processing and machine-learning kernels that must run at low energy cost. A growing share of these DSPs owe their efficiency to wide vector units: single-instruction, multiple-data (SIMD) hardware in which one instruction operates on an entire register of parallel data lanes at once, rather than on a single value. A DSP that issues one instruction across, say, 32 lanes can in principle retire 32 arithmetic operations per cycle instead of one, provided the surrounding code can keep those lanes fed with contiguous data. As DSPs have grown these wide vector units, dense matrix multiplication (GEMM), the compute core of most signal-processing and machine-learning workloads, has become the natural kernel to map onto them.

A developer who sets out to do this faces three questions. How much of the achievable performance does vectorization alone deliver? The DSP also offers hardware multithreading, so what does a second axis of parallelism add on top? And is any of this effort worthwhile when the vendor already ships an optimized GEMM in its SDK? The open literature offers surprisingly little guidance: papers rarely benchmark a hand-written kernel against a vendor's shipped implementation under controlled, matched conditions, even on platforms where that implementation's source ships in the SDK alongside the prebuilt binary.

A fourth question sits underneath the other three. Results in this space are usually reported from cycle-accurate simulators, and it is fair to ask whether they survive contact with silicon, where cache state, dynamic voltage and frequency scaling (DVFS), memory bandwidth, and the cost of getting data to the accelerator all intrude.

This paper answers all four questions empirically on the Hexagon DSP. We implement six single-precision GEMM variants that cross two execution modes, scalar (sequential) and vector (SIMD), with three threading policies, single-threaded, statically scheduled multithreaded, and dynamically scheduled multithreaded. Each variant is offloaded from the host CPU to the DSP and timed with on-chip performance counters, allowing a direct, cycle-accurate comparison against the vendor GEMM library across varies parameters like matrix size, thread count, etc.

The results answer each question clearly. First, a small hand-written vector kernel matches the vendor library's throughput at large matrix sizes. This shows that most of the library's advantage comes from its SIMD mapping and memory layout, not from hidden proprietary tuning. Second, the vendor library runs on a single thread. When we add multithreading to our own vector kernel, it beats the library by up to $2.6\times$ at the largest matrix size we measure. The vector speedup from threading saturates at exactly the number of concurrent vector units the hardware provides – six on this SoC – while dynamic scheduling proves consistently more robust than static scheduling.

Concretely, we contribute (i) a controlled six-variant implementation of Hexagon GEMM spanning scalar and vector execution, with the vector kernel further explored under single- and multi-threaded execution and static and dynamic block scheduling (Section III); (ii) an on-device measurement study

TABLE I
ABBREVIATIONS USED IN THIS PAPER.

Term	Meaning
DSP	Digital signal processor
SoC	System-on-chip
SIMD	Single instruction, multiple data
VLIW	Very long instruction word
GEMM	General matrix-matrix multiply (Level-3 BLAS)
HLOS	Host high-level OS; the CPU side of an offload
CDSP	Compute DSP; the Hexagon accelerator core
HVX	Hexagon Vector eXtensions (the vector unit)
QuRT	On-chip real-time OS / thread scheduler
HVX ctx	Vector-issue context (2 on the QCS6490)
VTCM	Vector tightly-coupled memory (scratchpad)
qf32	HVX vector single-precision format
L1D	L1 data cache (private to the scalar core)
Pcycles	Processor cycles (timing metric)
PMU	Performance-monitoring unit (on-chip counters)
DVFS	Dynamic voltage and frequency scaling
FastRPC	CPU↔DSP remote-procedure-call offload
skel	FastRPC remote stub exposing DSP-side methods
PD	Protection domain (signed or unsigned)
ION/`rpcmem`	Zero-copy shared-memory buffer allocator
SAM	Static Assignment Method (scheduling)
DAM	Dynamic Assignment Method (scheduling)
QHL	The vendor SDK math library (qhblas)
ISA	Instruction set architecture

against the vendor library across matrix size, tile size, and thread count, together with an end-to-end CPU-versus-offload comparison (Section IV); and (iii) actionable findings for practitioners: match the library first and then thread past it, expect vector scaling to stop at the hardware's concurrent-context count, prefer dynamic scheduling, keep the block size at or above the SIMD width to avoid leaving lanes idle, and offload only vectorized work.

II. Background and System Model

Table I collects the abbreviations used throughout; the concepts behind them are introduced below.

A. The Hexagon DSP

The Hexagon DSP has evolved through several architectural generations, and the device used for our experiments, the RUBIK Pi 3, integrates version v68 of this architecture [16]. RUBIK Pi 3 is a compact edge-AI development platform built around the Qualcomm Dragonwing QCS6490 system-on-chip (SoC) [17], which combines a Kryo 670 CPU, an Adreno 643 GPU, and a Hexagon DSP.

The Hexagon is a very-long-instruction-word (VLIW) scalar core processor [6], in which the compiler schedules multiple instructions to issue together each cycle. This single scalar core achieves parallelism through hardware multithreading, managed by the POSIX-like QuRT real-time OS [2]. The compute DSP on the QCS6490 provides six such hardware threads.

Alongside the scalar core, Hexagon v68 provides a coprocessor called the Hexagon Vector eXtensions (HVX), which implements the architecture's SIMD capability [1]. HVX comprises of its own vector registers and dedicated vector compute

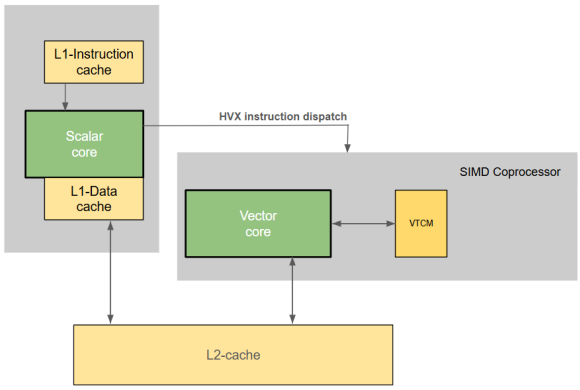

Fig. 1. Memory organisation of the Hexagon DSP. The scalar core owns private L1 instruction and data caches, while the HVX coprocessor reads and writes through its own path and a dedicated VTCM scratchpad; both sides meet at the shared L2 cache, which serves as the vector unit's first-level memory.

elements, which are collectively referred to as an *HVX vector context*. A hardware thread must dynamically attach itself to a *vector context* before it can issue HVX instructions. Crucially, Hexagon offers only a small fixed number of *HVX vector contexts*. QCS6490 supports two such *vector contexts*. So at most two hardware threads can execute HVX code simultaneously, while the remaining threads can continue executing scalar-only instructions. Each HVX vector register is 1024 bits (128 bytes) wide, giving a 1024-bit SIMD lane per active *vector context*. All HVX computation is carried out through a distinct HVX instruction subset that operates on these vector registers, separate from the base scalar ISA, though HVX and scalar instructions may be co-issued within the same VLIW packet. HVX supports both integer and floating-point operations.

Figure 1 shows the memory system, which shapes every result in Section IV. The scalar core is fed by private L1 instruction and data caches. The vector coprocessor bypasses L1 entirely: its loads and stores are served by the shared L2 and in addition to this shared cache path, HVX has access to a local scratch memory called the Vector Tightly Coupled Memory (VTCM) [8]. Because VTCM is a dedicated, directly-addressed memory rather than a cache, data placed there is not subject to eviction the way an L2 cache line can be when its set is full, and access to it avoids the tag-check and associativity overhead of a cache lookup, making it faster than L2.

B. GEMM and vectorisation

GEMM, the general matrix-matrix multiplication $C = A \times B$, is a very popular routine with several applications. This paper studies single-precision floating-point GEMM. In the conventional triple-loop implementation of GEMM, the innermost loop computes the dot product between one row of A and one column of B.We vectorise this reduction using HVX, which performs the same operation on 32 single-precision floating-point values simultaneously. When the operands are stored contiguously in memory, each vector

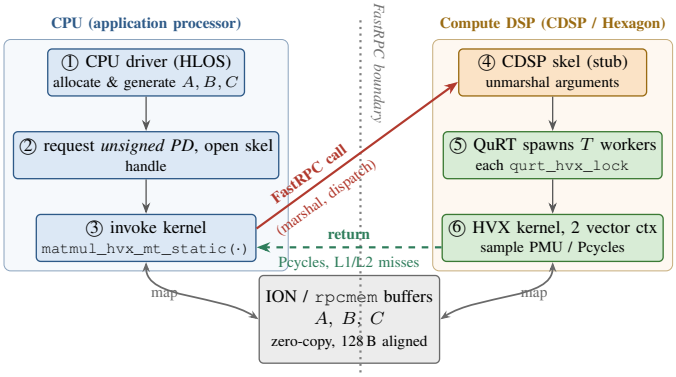

Fig. 2. One FastRPC offload from the CPU to the Hexagon compute DSP (CDSP). The host allocates shared `rpcmem`/ION buffers, opens the DSP skel, and invokes the kernel through FastRPC. On the DSP, the skel unmarshals the arguments, QuRT launches worker threads, and each worker acquires an HVX context before executing the vector kernel. Since the input and output buffers are shared between the CPU and DSP, data is transferred without copying, and only the performance counters are returned to the host.

instruction replaces up to 32 scalar operations. SIMD thus provides data-level parallelism, while QuRT threads provide thread-level parallelism. This paper evaluates the contribution of each independently and in combination.

C. The CPU–DSP boundary

The Hexagon compute DSP (CDSP) cannot be accessed directly from host applications. Instead, the CPU invokes DSP functions through FastRPC (*Fast Remote Procedure Call*), Qualcomm's remote procedure call framework. The DSP application is compiled as a *skel* (skeleton) binary that exports the DSP-side functions. The host loads this binary and invokes its functions transparently through FastRPC.

Input and output buffers are allocated using the `rpcmem`/ION shared-memory allocator, making them accessible to both the CPU and the DSP. Consequently, data is exchanged without copying across the CPU–DSP boundary because only references to the shared buffers are passed. The DSP application executes inside an unsigned protection domain (PD). A protection domain is an isolated user-space execution environment on the DSP with restricted privileges. Unsigned protection domains allow developer-built DSP applications to run safely without access to privileged system resources. Figure 2 illustrates the complete execution flow.

III. Algorithm Description

Figure 3 summarizes the design end to end. The product C is tiled into $k \times k$ blocks. The main function of the algorithm called driver spawns T QuRT workers. Each worker thread th_i tries to acquire an HVX context. If successful, th_i runs vector code. Otherwise, th_i executes the scalar code on failure by picking blocks under one of the two policies (described below). Table II lists the six variants we benchmark including the vendor provided baseline. QHL (qhblas) ships with full source in the Hexagon SDK.

TABLE II
The six benchmarked variants plus the vendor baseline.

Variant	Description
Seq	Single-threaded scalar triple-loop baseline.
Static	Scalar multithreaded; SAM block assignment.
Dynamic	Scalar multithreaded; DAM (atomic) block assignment.
Hvx	Single-threaded HVX, 32 floats/vector.
Hvx+Mt(Dam)	HVX + DAM (dynamic) multithreading.
Hvx+Mt(Sam)	HVX + SAM (static) multithreading.
Qhl	Vendor SDK library (qhblas), single-threaded.

A. Scalar baselines and the vector inner kernel

The scalar variants implement the conventional three-nested-loop matrix multiplication. Seq executes the entire computation sequentially, while Static and Dynamic parallelize over matrix blocks using static and dynamic scheduling, respectively. In all three cases, the inner reduction loop walks down a column of B, resulting in strided memory accesses with poor spatial locality. As the matrix size exceeds the L2 cache capacity, these accesses generate frequent cache misses, which is reflected in the measurements presented in Section IV-B.

The HVX kernel (Algorithm 1) improve both memory locality and computation throughput. It first calls HvxLock, which reserves a 128B vector context (line 3); if none is available it returns immediately so the caller can fall back to scalar code. For each output row i and vector-width block of columns starting at j, the reduction **for** loop over p (line 10). Within it, Broadcast replicates the scalar $A[i][p]$ into all 32 lanes (line 11); VLoad issues one aligned 128B read (load) of 32 contiguous elements from row p of B (line 12); QMul multiplies the two vectors lane-wise, resulting in a `qf32` product vector (line 13); and QAdd adds that product into the running `qf32` accumulator (line 14). That accumulator is cleared once per column block by VZero, which produces an all-zero `qf32` register (line 9), and holds 32 partial sums of row i of C simultaneously. After the reduction completes, ToSf converts the `qf32` accumulator to IEEE single precision (line 16) and VStore writes the 32 results back with one aligned 128B store (line 17). Accessing B through these contiguous, vector-aligned VLoads (line 12) substantially improves cache locality compared with the scalar implementations, while any remaining columns that do not fill a complete vector are finished by the scalar cleanup loop, one Dot product per element (line 19).

B. Per-block computation and the tile-width condition

The multithreaded HVX variants perform the same computation as the single-threaded kernel, but process one $k \times k$ tile at a time instead of entire row bands. This introduces one additional constraint: the tile must be wide enough for vector execution. As shown in Algorithm 2, the kernel first computes the tile width $w = c_1 - c_0$ (line 2), then $vecCols = \lfloor w/32 \rfloor$, the count of complete 32-lane HVX vectors the tile holds

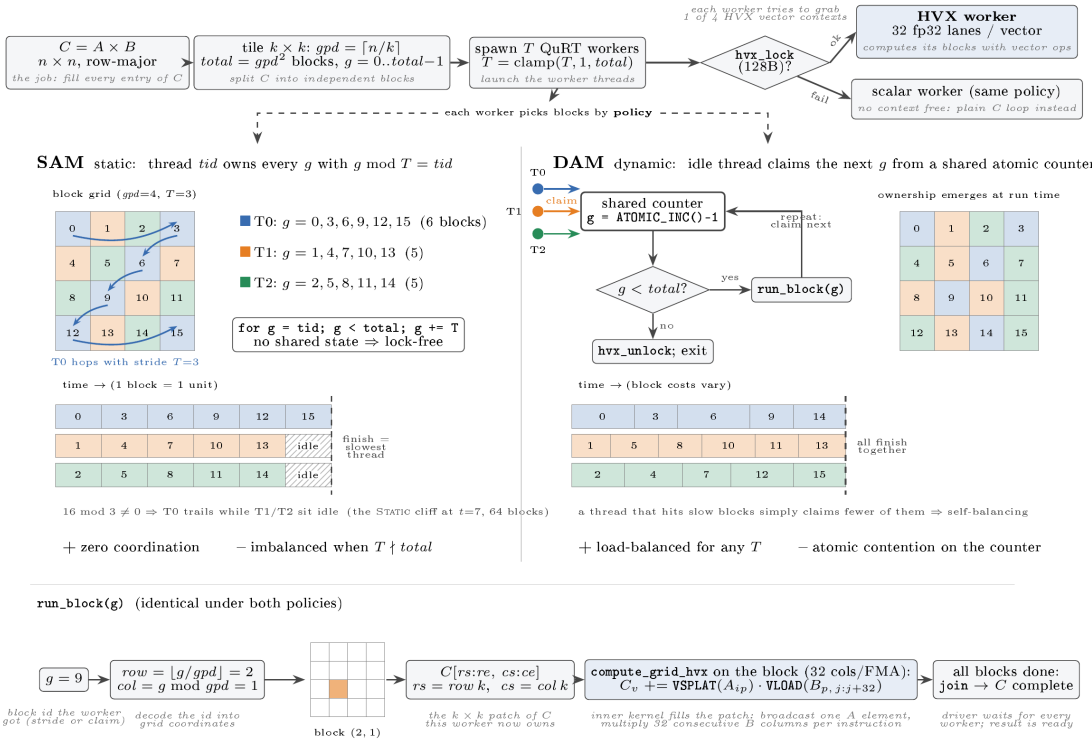

Fig. 3. End-to-end execution flow of the multithreaded HVX GEMM. Top: the input matrix is partitioned into $k \times k$ blocks, QuRT worker threads are spawned, and each acquires an HVX context or falls back to the scalar kernel if none is available. Middle: block scheduling with static assignment (SAM) and dynamic assignment (DAM). SAM assigns fixed blocks to workers, whereas DAM balances the workload by dynamically assigning blocks from a shared atomic counter. Bottom: each block is processed by the same HVX kernel, where a linear block index is mapped to its corresponding tile and computed 32 columns at a time.

(line 3); any remaining columns are left to the scalar tail, which finishes each with a single DOT product (line 18). The vector **for** loop (line 6) runs exactly *vecCols* times, so when the tile edge k falls below 32 no complete vector fits inside a tile: the loop executes zero iterations, the vectorised BROADCAST/VLOAD/QMUL/QADD body is skipped entirely, and every element is instead produced by the scalar-tail DOT loop (line 18). Consequently, a kernel that is otherwise vectorised executes purely as scalar code. This behaviour is determined solely by the tile geometry, not by the scheduling policy, and is the reason for the performance cliff observed in Section IV-C.

C. Workers, scheduling, and the driver

The two multithreaded HVX variants differ only in how work is assigned to threads (Algorithms 3 and 4). In the static assignment model (SAM, HVXWORKERSAM, Algorithm 3), each worker owns a fixed strided subset of blocks, $\{tid, tid{+}T, tid{+}2T, \dots\}$: the strided **for** loop (line 6) advances the block index by T each step, so no shared scheduling state is needed. In the dynamic assignment model (DAM, HVXWORKERDAM, Algorithm 4), each worker instead claims its next block with ATOMICINC, an atomic post-increment of the shared counter that hands back a unique index (line 7), and **break**s once that index passes the last block (line 8).

Before executing a block, each worker calls HVXLOCK to reserve a 128 B HVX context (line 2 in Algorithm 3, line 2 in Algorithm 4). If the reservation fails, the worker delegates the identical computation to its scalar counterpart, SCALARWORKERSAM or SCALARWORKERDAM (line 3 and line 3, respectively). This fallback guarantees that all work is completed, although without the benefit of vector execution.

IV. EXPERIMENTS AND RESULTS

A. Experimental setup

All measurements were taken on a Thundercomm Rubik Pi 3 single-board computer built around the Qualcomm QCS6490 SoC. The host is an octa-core Kryo 670 CPU (one Cortex-A78 at 2.7 GHz, three Cortex-A78 at 2.4 GHz, and four Cortex-A55 at 1.9 GHz), which also runs our CPU baseline. The accelerator is the QCS6490 compute DSP, a Hexagon v68 target with six hardware threads and two HVX contexts. The board runs Qualcomm Linux (Ubuntu 24.04), and all DSP kernels are compiled with `hexagon-clang -mv68 -O2 -ffast-math` from the Hexagon SDK.

The kernels are invoked from a CPU-side driver through FastRPC. The driver allocates the input and output matrices

The source code is available in the https://anonymous.4open.science/r/VecMatMul-F05C/README.md repository.

Algorithm 1 Single-threaded HVX inner kernel.

procedure MATMULHVX(A, B, C, n)
  $vecCols \leftarrow \lfloor n/32 \rfloor$ //full 32-wide groups
  if not HVXLOCK(128B) **then**
    return //no vector context
  end if
  for $i \leftarrow 0$ **to** $n-1$ **do**
    for $jb \leftarrow 0$ **to** $vecCols-1$ **do**
      $j \leftarrow jb \cdot 32$
      $\mathbf{acc} \leftarrow$ VZERO //`qf32` accumulator, 32 lanes
      for $p \leftarrow 0$ **to** $n-1$ **do**
        $\mathbf{a} \leftarrow$ BROADCAST($A[i][p]$)
        $\mathbf{b} \leftarrow$ VLOAD($B[p][j:j{+}31]$)
        $\mathbf{prod} \leftarrow$ QMUL($\mathbf{a}, \mathbf{b}$)
        $\mathbf{acc} \leftarrow$ QADD($\mathbf{acc}, \mathbf{prod}$)
      end for
      $\mathbf{sf} \leftarrow$ TOSF($\mathbf{acc}$)
      VSTORE($C[i][j:j{+}31],\ \mathbf{sf}$)
    end for
    for $j \leftarrow vecCols \cdot 32$ **to** $n-1$ **do** //scalar tail, only if $32 \nmid n$
      $C[i][j] \leftarrow$ DOT($A[i][:],\ B[:][j]$)
    end for
  end for
  HVXUNLOCK
end procedure

Algorithm 2 COMPUTEGRIDHVX: per-block HVX kernel

procedure COMPUTEGRIDHVX($A, B, C, n, r_0, r_1, c_0, c_1$)
  $w \leftarrow c_1 - c_0$
  $vecCols \leftarrow \lfloor w/32 \rfloor$ //full 32-lane vectors in tile
  $tail \leftarrow c_0 + 32 \cdot vecCols$
  for $i \leftarrow r_0$ **to** $r_1 - 1$ **do**
    for $jb \leftarrow 0$ **to** $vecCols-1$ **do** //0 iters when $k < 32$
      $j \leftarrow c_0 + 32 \cdot jb$
      $\mathbf{acc} \leftarrow$ VZERO //`qf32` accumulator
      for $p \leftarrow 0$ **to** $n-1$ **do**
        $\mathbf{a} \leftarrow$ BROADCAST($A[i][p]$)
        $\mathbf{b} \leftarrow$ VLOAD($B[p][j:j{+}31]$)
        $\mathbf{prod} \leftarrow$ QMUL($\mathbf{a}, \mathbf{b}$)
        $\mathbf{acc} \leftarrow$ QADD($\mathbf{acc}, \mathbf{prod}$)
      end for
      $\mathbf{sf} \leftarrow$ TOSF($\mathbf{acc}$)
      VSTORE($C[i][j:j{+}31],\ \mathbf{sf}$)
    end for
    for $j \leftarrow tail$ **to** $c_1 - 1$ **do** //scalar cleanup
      $C[i][j] \leftarrow$ DOT($A[i][:],\ B[:][j]$)
    end for
  end for
end procedure

Algorithm 3 HVX worker loop with static block assignment (SAM).

procedure HVXWORKERSAM(ctx, tid)
  if not HVXLOCK(128B) **then**
    SCALARWORKERSAM(ctx, tid)
    return
  end if
  for $g \leftarrow tid$ **to** $ctx.totalGrids - 1$ **step** $ctx.T$ **do**
    $(r_0, r_1, c_0, c_1) \leftarrow$ BLOCKBOUNDS(g, ctx)
    COMPUTEGRIDHVX($\ldots, r_0, r_1, c_0, c_1$)
  end for
  HVXUNLOCK
  THREADEXIT
end procedure

Algorithm 4 HVX worker loop with dynamic block assignment (DAM).

procedure HVXWORKERDAM(ctx)
  if not HVXLOCK(128B) **then**
    SCALARWORKERDAM(ctx)
    return
  end if
  loop
    $g \leftarrow$ ATOMICINC($ctx.counter$) $- 1$
    if $g \geq ctx.totalGrids$ **then**
      break
    end if
    $(r_0, r_1, c_0, c_1) \leftarrow$ BLOCKBOUNDS(g, ctx)
    COMPUTEGRIDHVX($\ldots, r_0, r_1, c_0, c_1$)
  end loop
  HVXUNLOCK
  THREADEXIT
end procedure

(A, B, and C) as `rpcmem`/ION shared-memory buffers, requests an unsigned protection domain, opens a single DSP skel, and invokes the selected kernel. Because the buffers are shared between the CPU and DSP, the operands are accessed zero-copy after mapping into the DSP address space. Their page alignment also satisfies the 128-byte alignment required by HVX vector loads. Every kernel is exposed as a separate remote method behind the same FastRPC handle, so all seven variants share an identical offload path and differ only in their compute kernel.

We report end-to-end wall-clock time, in milliseconds, as the primary metric because it reflects the latency observed by the caller and includes the full FastRPC overhead. Each measurement begins immediately before the FastRPC call and ends after it returns. We also record the on-chip processor-cycle counter (`QDSP6_clk_cnt`, register `c15:14`) around the compute region, together with the Hexagon PMU configured to count L1D demand misses (event `0x11`) and L2 read misses (event `0x64`). The PMU counters are enabled before spawning worker threads and read after they join. Since the counters are

32-bit, L1D counts may wrap at the largest problem sizes and are therefore treated as indicative only. These hardware counters serve as a cross-check against the dispatch and scheduling noise present in wall-clock measurements. All reported results were validated against a sequential CPU reference using a checksum comparison,if checksum vary(because of floating-point operations) more than our threshold we declare it wrong.

Sathya: 1. What is absolute and relative tolerance? 2. Why are you counting only L2 read misses and not L2 write misses as well?

Reply: we are measuring L2 readmisses because our workload is read heavy

We evaluate three parameter matrix-size - n , tile-size - k , number of threads - t through our experiments. The matrix-size(n) experiment uses $n \in \{32, 64, 128, 256, 512, 1024, 2048\}$ with t=4 and k=32. The tile-size Experiment uses $k \in \{2, 4, 8, 16, 32, 64, 128, 256\}$ with n=1024 and t=4. The thread-scaling Experiment uses $t \in \{1, \ldots, 10\}$ with n=1024 and k=32. All inputs are dense. Configurations up to n=512 are repeated three times and reported as arithmetic means. The faster variants (HVX, HVX+MT(DAM), HVX+MT(SAM), and QHL) are also repeated three times at n=1024. The three scalar variants at n=1024 and all variants at n=2048 are measured once because the scalar reference validation becomes prohibitively time consuming at those sizes. Section IV-G reports the observed run-to-run variation. The repository linked on the first page contains the kernels, benchmarking harness, and raw measurement data used to generate all reported tables and figures.

Sathya: 1. What is a sweep? 2. Did you explain n, t, k elsewhere? 3. Is the defn of scalar variant explained?

Reply: replaced word sweep with experiment , parameters are explained now , scaler variants in explained in table 2

B. Scaling with matrix size

Table III and Fig. 4 show how execution time changes with matrix size. Two performance bands emerge clearly. The scalar variants form the slow band, while the vectorised variants and the vendor library cluster together in a much faster band. As the matrices grow, the gap between the two bands widens substantially: at n=256, SEQ is already 61× slower than HVX, and the difference continues to increase for larger matrices. The reason is straightforward. The scalar kernels traverse columns of B with poor spatial locality, generating increasing cache misses as the working set outgrows the cache, whereas the HVX kernels stream contiguous vectors from memory and maintain much better locality.

Within the vector band, multithreading begins to pay off once the problem is large enough to amortise its overhead. From n=128 onward, HVX+MT(DAM) consistently outperforms the single-threaded HVX kernel, providing a speedup of

More experiments results are available in https://anonymous.4open.science/r/VecMatMul-F05C/extended-hipc-paper-2026.pdf.

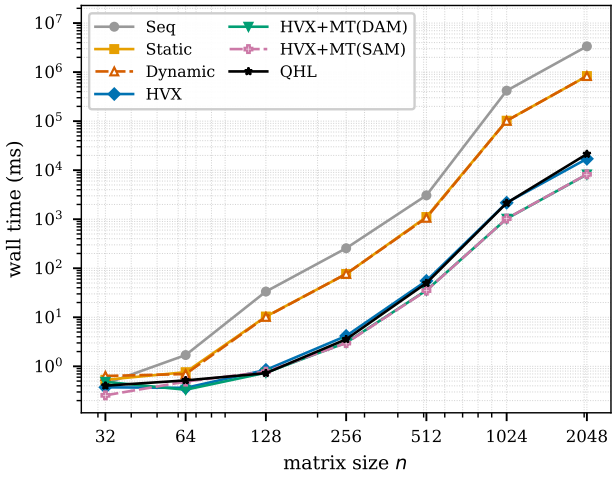

Fig. 4. Wall-clock time vs. matrix size (log–log). The scalar variants form a slow band; the vector variants and QHL cluster in a fast band. HVX+MT(DAM) leads the fast band for $n \geq 128$. Ranking below n=128 is noisy; see Section IV-G.

1.1–2.1×. At the largest problem size (n=2048), it completes in 8.2 s compared with 21.4 s for the vendor library QHL.

For the smallest matrices (n=32 and 64), the wall-clock ordering is less stable because the FastRPC round trip is comparable to, or larger than, the DSP computation itself. In this regime, small variations in dispatch latency can dominate the measured execution time.

The L2 read-miss counts (Fig. 8) explain why the bands separate at all. At n=1024 the scalar SEQ kernel takes 66M L2 read misses as its working set overflows the cache, while the vector variants and QHL stay two to three orders of magnitude lower; by n=2048 the scalar misses have grown to 527M. The vector kernels keep their B access unit-stride and vector-aligned, so they stay cache-resident where the scalar loop cannot, and HVX+MT(DAM) actually incurs fewer L2 misses than single-threaded HVX at the largest sizes because tiling improves reuse. This locality story, measured on the on-chip counters, is what the wall-clock bands in Fig. 4 ultimately trace back to.

C. Sensitivity to tile size

The tile(k) Experiment (Fig. 5) at n=1024, wall time tracks compute cleanly. For the scalar STATIC and DYNAMIC variants the tile edge barely matters; both hover near 105 s at every k, since the scalar loop does not vectorise and k only sets load-balance granularity. SEQ, HVX, and QHL do not use tiling at all, so we plot each as a flat reference line. The two multithreaded HVX variants behave bimodally. From k=32 upward they are flat and fast, within 2.5% of one another and best at k=256 (1.01 s). Below k=32 they collapse by roughly 207×, to about 209 s, landing slower than even the scalar band.

The mechanism is the tile-width condition of algorithm 2. With $k \in \{2, 4, 8, 16\}$ no 32-lane group fits inside a block, so `vec_cols` is zero and every column is computed in the

TABLE III

WALL-CLOCK TIME (MS) VS. MATRIX SIZE n (t=4, k=32). BRACKETS: SPEEDUP OVER SEQ. VALUES ARE PER-RUN MEANS ($n \leq 512$) OR SINGLE RUNS ($n \geq 1024$). RANKINGS BELOW n=128 CARRY SUBSTANTIAL DISPATCH NOISE; SEE SECTION IV-G.

n	SEQ	STATIC	DYNAMIC	HVX	HVX+MT(DAM)	HVX+MT(SAM)	QHL
32	0.440	0.531 (0.8×)	0.636 (0.7×)	0.373 (1.2×)	0.483 (0.9×)	0.256 (1.7×)	0.403 (1.1×)
64	1.697	0.759 (2.2×)	0.696 (2.4×)	0.364 (4.7×)	0.333 (5.1×)	0.488 (3.5×)	0.516 (3.3×)
128	33.47	10.36 (3.2×)	10.22 (3.3×)	0.845 (39.6×)	0.740 (45.2×)	0.777 (43.1×)	0.725 (46.2×)
256	255.9	76.39 (3.4×)	76.83 (3.3×)	4.163 (61.5×)	3.036 (84.3×)	2.978 (85.9×)	3.558 (71.9×)
512	3,066	1,109 (2.8×)	1,068 (2.9×)	55.27 (55.5×)	35.42 (86.6×)	34.99 (87.6×)	49.83 (61.5×)
1024	418,419	102,033 (4.1×)	101,631 (4.1×)	2,168 (193.0×)	1,023 (409.0×)	1,029 (406.5×)	2,142 (195.4×)
2048	3,355,757	829,301 (4.1×)	831,848 (4.0×)	17,074 (196.5×)	8,183 (410.1×)	8,240 (407.3×)	21,360 (157.1×)

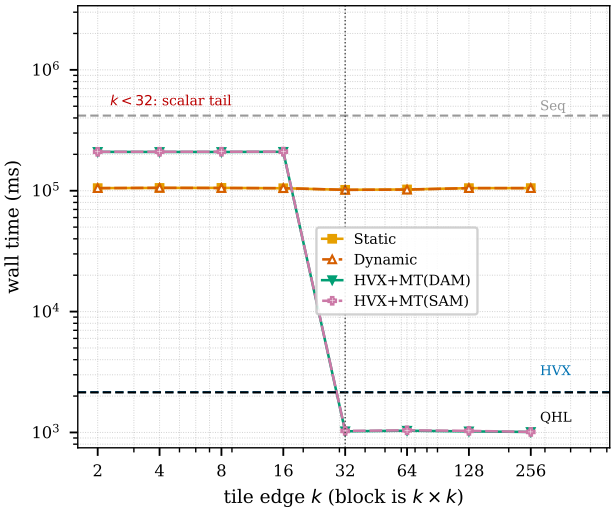

Fig. 5. Tile-size cliff, wall-clock time (n=1024, log–log). HVX+MT(DAM) and HVX+MT(SAM) lose two orders of magnitude below the 32-lane vector width (dotted line): a block narrower than a vector never issues a vector instruction. Above k=32 they are flat. Scalar STATIC and DYNAMIC are tile-insensitive throughout. SEQ, HVX, and QHL do not use tiling at all; their single measured values are shown as horizontal reference lines.

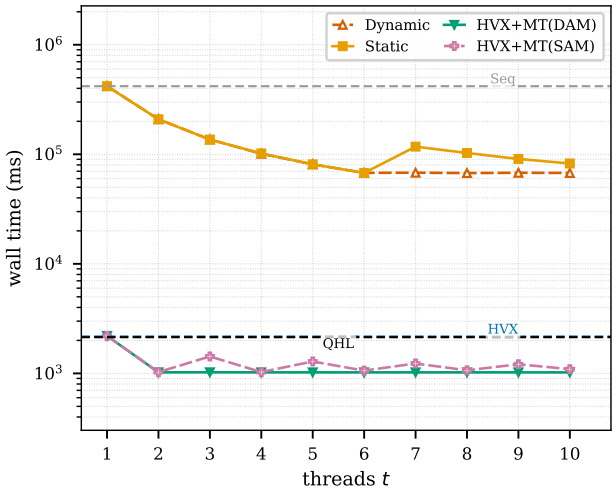

Fig. 6. Thread scaling, wall-clock time (n=1024, k=32, log y). Scalar STATIC and DYNAMIC scale to $t \approx 6$ (six hardware threads); HVX+MT(DAM) and HVX+MT(SAM) saturate at t=2 (two HVX contexts). STATIC and HVX+MT(SAM) show the SAM load-imbalance cliff when t does not divide the 1024-block grid; DYNAMIC and HVX+MT(DAM) stay flat. SEQ, HVX, and QHL are thread-independent and shown as horizontal reference lines at their single measured value (t=4).

scalar tail while the vector unit sits idle. the tile edge must be at least the vector width, and nothing in the kernel's return value warns a caller when it is violated.

D. Scaling with thread count

The thread Experiment (Table IV, Fig. 6), again at n=1024, separates the two kinds of parallelism cleanly, and both saturation points land exactly where the hardware says they should. The scalar variants scale almost linearly to six threads, where DYNAMIC reaches 6.2× and flattens: six is the number of hardware threads on this DSP, and a seventh software thread has nothing to run on. The vector variant stops much earlier. HVX+MT(DAM) gains 2.15× going from one thread to two and is then flat all the way to t=10, because the QCS6490 has two HVX contexts. The first two workers take them; a third worker's `hvx_lock` fails and it drops to the scalar path, contributing almost nothing next to the vector threads. The near-ideal doubling from one context to two says each context is fully effective, which rules out memory bandwidth or issue width as the ceiling. The ceiling is simply the context count.

Scheduling shows its worth under imbalance. At n=1024, k=32 the grid holds 1024 blocks. Scalar STATIC climbs smoothly to t=6 and then hits a cliff at t=7, jumping from 67.6 s to 117.5 s because 1024 does not divide by 7, and it stays ragged from there, while DYNAMIC holds flat. The same pattern repeats in the vector band: HVX+MT(SAM) oscillates between about 1.03 and 1.43 s as t moves through values that do and do not divide the block count, whereas HVX+MT(DAM) sits at about 1.02 s throughout. The atomic counter buys a schedule that simply does not care how the block count factorises. Its cost is negligible at this granularity: at the thread counts where SAM has no imbalance, HVX+MT(DAM) and HVX+MT(SAM) agree within 0.7% at t=1, 2, 4, widening to 4.4% at t=8; that residual is dispatch-level wall-clock noise rather than genuine contention, since the on-chip Pcycles counter shows the two tracking within 0.7% across the board (Section IV-G). Either way, DAM's advantage over SAM is robustness, not raw speed.

TABLE IV
WALL-CLOCK TIME (MS) VS. THREAD COUNT t (n=1024, k=32). SEQ, HVX, AND QHL ARE THREAD-INDEPENDENT REFERENCES (418,471, 2,171, 2,140 MS). BRACKETS: SPEEDUP OVER SEQ.

t	STATIC	DYNAMIC	HVX+MT(DAM)	HVX+MT(SAM)
1	418,667 (1.0×)	418,857 (1.0×)	2,201 (190.2×)	2,199 (190.4×)
2	208,874 (2.0×)	208,973 (2.0×)	1,022 (409.5×)	1,028 (407.3×)
3	136,278 (3.1×)	136,498 (3.1×)	1,023 (409.2×)	1,428 (293.3×)
4	101,913 (4.1×)	101,206 (4.1×)	1,022 (409.5×)	1,029 (406.7×)
5	80,994 (5.2×)	80,996 (5.2×)	1,022 (409.5×)	1,285 (325.7×)
6	67,555 (6.2×)	67,661 (6.2×)	1,022 (409.6×)	1,060 (394.9×)
7	117,456 (3.6×)	67,928 (6.2×)	1,022 (409.5×)	1,231 (340.0×)
8	102,807 (4.1×)	67,389 (6.2×)	1,022 (409.6×)	1,067 (392.3×)
9	90,675 (4.6×)	67,799 (6.2×)	1,022 (409.5×)	1,212 (345.3×)
10	82,470 (5.1×)	67,562 (6.2×)	1,021 (409.7×)	1,093 (382.9×)

E. Comparison with the vendor library

Figure 7 plots every hand-written variant's wall-clock speedup over QHL, including the scalar ones, which sit two to three orders of magnitude below the library throughout and mainly confirm the comparison worth making is among the vector variants. Below n=128 the ranking is dispatch noise, not signal (Section IV-B), so we anchor from n=128 upward. There, single-threaded HVX trails the library by 1.1 to 1.2× through n=512, reaches near parity at n=1024 (0.99×), and is 1.25× ahead at n=2048. So a compact hand-written vector kernel reaches, and at scale exceeds, the library's single-thread throughput. Where it matters, at large n, the SIMD mapping and locality do the work; there is no sign of proprietary tuning the kernel cannot match.

The multithreaded variants carry the headline. HVX+MT(DAM) and HVX+MT(SAM) sit just under parity with the library at n=128 (0.98× and 0.93×), cross it by n=256 (1.17× and 1.20×), and climb monotonically to 2.61× and 2.59× at n=2048, with the dynamic and static policies tracking each other closely throughout. Below the crossover the library wins, since threading overhead dominates a tiny problem. What makes the result notable is where the gain comes from: the single-threaded kernel already matches QHL, so every additional factor is thread-level parallelism, a lever the library simply does not use, laid over an unchanged inner kernel.

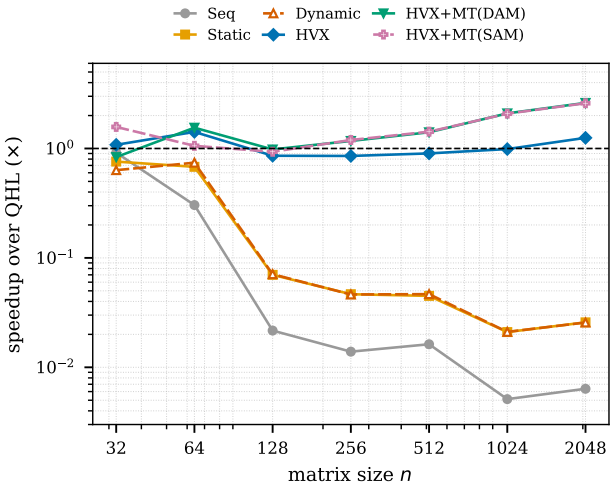

Fig. 7. Wall-clock speedup over the vendor library QHL vs. n (log–log), all six hand-written variants. The scalar SEQ, STATIC, and DYNAMIC sit two to three orders of magnitude below QHL throughout. Single-thread HVX trails through n=512, reaches near parity at n=1024, and overtakes QHL at n=2048. HVX+MT(DAM) and HVX+MT(SAM) cross QHL by n=256 and reach 2.6× at n=2048. Ranking below n=128 carries substantial dispatch noise; see Section IV-G.

F. CPU baseline versus DSP offload

Wall time so far has measured only the DSP side of the call. The question a system designer actually faces is different: given a GEMM running on the CPU, is it worth shipping to the DSP once the round trip is paid? To answer it we ran the same naive triple-loop on the host CPU, single-threaded and multithreaded, and compared its end-to-end wall time against offloading each DSP variant over FastRPC. Every number in Table V and Fig. 9 includes the full round trip, so the offload cost is already inside the DSP figures. Three effects separate cleanly.

The first is a fixed floor that CPU-native code never pays: at n=32 the CPU finishes in about 0.08 ms, while every offloaded variant sits near 0.2 to 0.5 ms regardless of DSP compute, the same FastRPC floor identified in Section IV-B. For tiny GEMM, up to about n=64, the CPU wins outright since there is not enough work to amortise the trip.

The second is that offloading the scalar kernel is a net loss. The DSP's scalar throughput sits far below the CPU's, so the identical multithreaded scalar loop runs 3.2× slower on the DSP at n=256 and 11.4× slower at n=512. Shipping the loop as-is to the accelerator is the wrong move. The DSP earns its place through the vector unit alone: on the same DSP, the vector kernel outruns the scalar one by 26× at n=512.

The third is that offloading the vectorised kernel pays off decisively at scale. HVX+MT(DAM) overtakes the best CPU variant from about n=96, is 7.8× faster at n=256, and 3.6× faster at n=2048 (7.5 s against 27.4 s); even the vendor library on the DSP beats the CPU at the largest size. Note that the system-level crossover, about n=96, differs from the DSP-only crossover precisely because of the offload floor.

These three effects compose into a simple additive picture: end-to-end cost is a fixed round-trip floor plus DSP compute that grows as n^3. Each doubling of n multiplies the

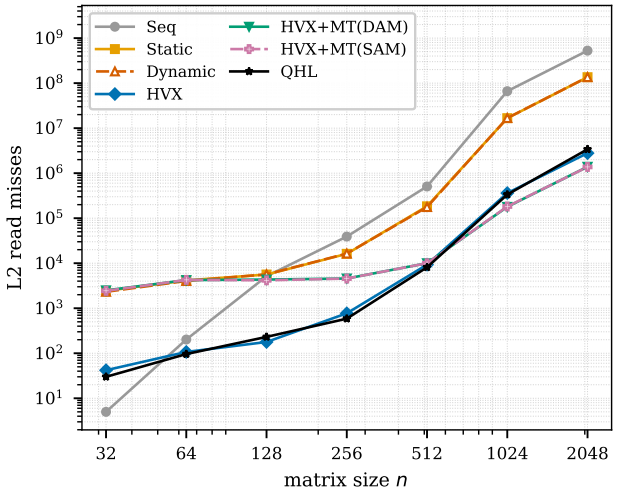

Fig. 8. L2 read misses vs. n (log–log), all seven variants. The scalar band (SEQ, STATIC, DYNAMIC) overflows L2 (SEQ reaches 5.3×10^8 misses at n=2048); the vector variants (HVX, HVX+MT(DAM), HVX+MT(SAM)) and QHL stay two to three orders of magnitude lower, which is the source of the band split in Fig. 4.

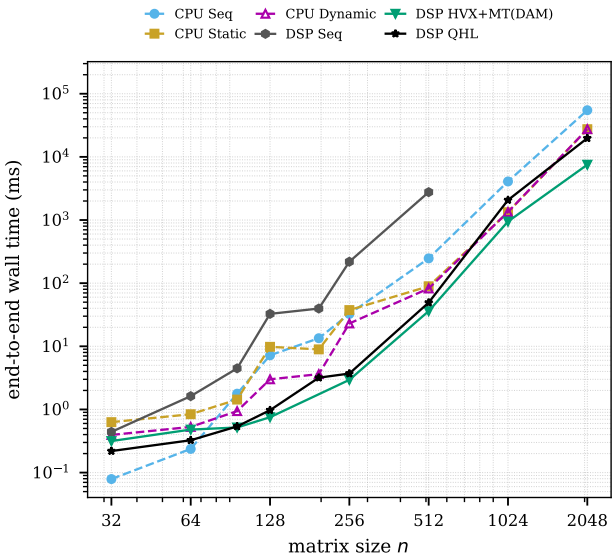

Fig. 9. End-to-end wall time vs. n (log–log), six representative series: three CPU-native variants (dashed) and three DSP-offloaded variants (solid): scalar SEQ as a reference, the winning HVX+MT(DAM), and the vendor QHL. Offloading the scalar loop to the DSP is slower than running it on the CPU (upper dashed cluster versus the DSP-SEQ line just below); offloading a vectorised kernel wins from $n \approx 96$ and reaches $3.6\times$ over the best CPU variant at n=2048. Below n=64 the FastRPC floor lets the CPU win regardless of variant.

TABLE V
END-TO-END WALL TIME (MS, CPU-SIDE ROUND TRIP), REPRESENTATIVE VARIANTS. COLUMNS 4 TO 6 ARE DSP OFFLOAD; DSP SCALAR DYNAMIC IS NOT RUN PAST n=512. THE REMAINING VARIANTS (CPU AND DSP STATIC, DSP SEQ, DSP HVX ALONE, DSP HVX+MT(SAM)) FOLLOW THE SAME TWO BANDS AND ARE OMITTED HERE FOR SPACE; ALL TEN ARE PLOTTED IN FIG. 9.

n	CPU-SEQ	CPU-DYNAMIC	DYNAMIC	HVX+MT(DAM)	QHL
32	0.08	0.39	0.40	0.32	0.22
64	0.24	0.53	0.66	0.48	0.33
96	1.78	0.94	3.95	0.52	0.54
128	7.20	3.00	10.42	0.75	0.97
256	32.5	23.0	73.2	2.94	3.70
512	246.9	82.4	941.8	36.1	49.2
1024	4079	1339	n/a	947	2070
2048	54829	27467	n/a	7519	19770

compute term by eight, so the floor stops mattering within a doubling or two of wherever the two terms cross, which on this platform falls between n=64 and n=128, exactly where HVX+MT(DAM) pulls away from the CPU.

The omitted variants (Fig. 9) fall in the same two bands: CPU-STATIC tracks CPU-DYNAMIC, DSP STATIC tracks DYNAMIC, and HVX+MT(SAM) tracks HVX+MT(DAM), so the three effects are not an artefact of variant choice.

One caveat frames all three observations. The CPU baseline is the same naive triple-loop, not a NEON-vectorised or tuned BLAS kernel, so these numbers quantify the benefit of moving a naive dense GEMM to the DSP vector unit, end to end. An optimised CPU library would narrow the gap, and the truly symmetric comparison, vector unit against vector unit, is one we leave for future work.

G. Measurement stability

The wall-clock numbers above rest on repeated measurement, and their spread is worth stating rather than assuming, since wall time carries dispatch and scheduling noise that the on-chip counter does not. Across the 39 variant-size combinations run three times (all seven variants at $n \leq 512$, plus HVX, HVX+MT(DAM), HVX+MT(SAM), and QHL at n=1024), the wall-clock spread, $(\max - \min)/\text{mean}$, has a median of 39% at n=32, falling to 25% by n=64–128, then dropping sharply to 1.9% at n=256 and 1.4% at n=512; the tile and thread sweeps, both run at n=1024, sit in this same low-noise regime throughout, which is why Sections IV-C and IV-D do not need the caveats that Section IV-B does below n=128.

The on-chip Pcycles counter tells a different story at exactly the sizes where wall time is noisiest, and that contrast is the point: Pcycles spread has a median of 0.7% across the same 39 combinations and a single largest outlier of 6.5%, at n=32, with no variant ever changing rank between runs. Table VI gives both spreads side by side by size. The gap between the two at small n is not a contradiction; it says that the DSP's own compute is highly reproducible throughout, and essentially all of the wall-clock noise below n=128 is dispatch and scheduling variance sitting on top of it, external to the kernel itself.

The cache counters that feed the L2 locality story are, like Pcycles, largely free of this dispatch noise, with one exception: HVX+MT(DAM)'s L1D spread reaches 290% at n=1024, the signature of a 32-bit counter wrapping under accumulation rather than genuine variation, and QHL's L1D spread is 61 to 114% over the same range; the corresponding L2 spreads never exceed 3.6%, which is why Section IV-B leans on L2 rather than L1D.

TABLE VI
RUN-TO-RUN SPREAD, WALL TIME VS. ON-CHIP PCYCLES, ACROSS ALL VARIANTS MEASURED THREE TIMES AT EACH SIZE ($n \leq 512$); THE FOUR FAST VARIANTS REPEATED AT n=1024 SPREAD 2–6% IN WALL TIME AND 0.08–0.67% IN PCYCLES.

n	wall-time median	wall-time max	Pcycles median	Pcycles max
32	39%	103%	3.3%	6.5%
64	25%	59%	1.2%	2.9%
128	26%	35%	0.4%	2.1%
256	1.9%	11%	0.5%	1.7%
512	1.4%	5.2%	0.6%	4.7%

V. RELATED WORK

Vendor manuals document HVX [1] and QuRT [2] separately, and the SDK ships qhblas [3] with full source; yet controlled, on-silicon comparisons against such a library as an unmodified external baseline remain rare, a gap this paper addresses. Classical GEMM tuning work – Goto and van de Geijn's blocking anatomy [4], ATLAS's automated search [10], and BLIS's portable framework [11] – centres on cache blocking and register tiling, echoed in our L2-miss results (Section IV-B); a roofline analysis [5] would further sharpen the low arithmetic intensity we observe (Section VI-B). Mobile SIMD GEMM work targets low-precision inference instead: CMSIS-NN on Cortex-M [12], QNNPACK on NEON [13], the gemmlowp/Ruy quantisation scheme [14], and the Arm Compute Library [15] share our lane-matched-blocking concern but target core-bound, not HVX-context-bound, hardware. None isolates thread-level parallelism as an explicit variable against a single-threaded vendor ceiling on a context-limited DSP, which is this paper's contribution.

VI. DISCUSSION AND CONCLUSIONS

A. Summary of findings

Stepping back, the experiments answer the questions posed at the outset. A small vector kernel is enough to match the vendor library at scale, so the essential ingredients are the SIMD mapping and memory layout rather than anything proprietary. The cheapest large gain on this platform is orthogonal parallelism: the vendor GEMM is single-threaded, and threading a kernel that already matches it yields up to $2.6\times$ without touching the inner loop. That gain arrives in full with the second vector thread, because two is the number of HVX contexts on this part; the four remaining hardware threads can only do scalar work. Dynamic block scheduling costs one atomic counter and in exchange never trips over an awkward block count, where static assignment loses up to 40%. The tile geometry carries one hard rule: keep the tile edge at or above the vector width, or the kernel degrades to scalar execution at a cost of two orders of magnitude. Together with the end-to-end measurements, this gives a deployment rule: offload dense GEMM only when vectorised and large enough to clear the FastRPC round trip, roughly $n \gtrsim 96$ here, and never offload scalar loops.

B. Limitations and future work

The study has limits worth naming. It covers one SoC, square dense single-precision inputs, and cycles rather than energy; the largest two sizes rest on single runs, though Section IV-G shows the variant orderings are stable and all critical ratios come from back-to-back runs on the same device; and the CPU baseline is deliberately naive.

Since the thread count is capped by contexts, the productive next step is not more threads but higher arithmetic intensity per context. Each inner iteration of Listing 1 performs a 32-lane fused multiply-add, 64 flops, against a fresh 128-byte load of B with no reuse, an intensity near 0.5 flops per byte; a register-blocked kernel that reuses each loaded B vector across r rows of A raises this almost linearly in r, and quantifying how much of that headroom the memory system delivers, alongside VTCM staging, an energy measurement, and parts that expose four HVX contexts, is the immediate follow-on work. We also plan to grow the study beyond square single-precision GEMM into a broader suite of vector linear-algebra kernels whose runtime it dominates.

REFERENCES

[1] Qualcomm Technologies, Inc., *Hexagon V69 HVX Programmer's Reference Manual*, 2021.

[2] Qualcomm Technologies, Inc., *Hexagon SDK: QuRT RTOS User Guide*, 2021.

[3] Qualcomm Technologies, Inc., *Hexagon DSP Math Library (QHL / qhblas) User Guide*, 2021.

[4] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, 2008.

[5] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, 2009.

[6] J. A. Fisher, "Very long instruction word architectures and the ELI-512," in *Proc. Int. Symp. Computer Architecture (ISCA)*, 1983, pp. 140–150.

[7] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. San Francisco, CA, USA: Morgan Kaufmann, 2008.

[8] R. Banakar, S. Steinke, B.-S. Lee, M. Balakrishnan, and P. Marwedel, "Scratchpad memory: a design alternative for cache on-chip memory in embedded systems," in *Proc. Int. Symp. Hardware/Software Codesign (CODES)*, 2002, pp. 73–78.

[9] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. S. Duff, "A set of level 3 basic linear algebra subprograms," *ACM Trans. Math. Softw.*, vol. 16, no. 1, pp. 1–17, 1990.

[10] R. C. Whaley and J. J. Dongarra, "Automatically tuned linear algebra software," in *Proc. ACM/IEEE Conf. Supercomputing (SC)*, 1998.

[11] F. G. Van Zee and R. A. van de Geijn, "BLIS: a framework for rapidly instantiating BLAS functionality," *ACM Trans. Math. Softw.*, vol. 41, no. 3, pp. 14:1–14:33, 2015.

[12] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: efficient neural network kernels for Arm Cortex-M CPUs," *arXiv preprint arXiv:1801.06601*, 2018.

[13] M. Dukhan, Y. Wu, and H. Lu, "QNNPACK: open source library for optimized mobile deep learning," 2018. [Online]. Available: https://github.com/pytorch/QNNPACK

[14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference,"

in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.

[15] Arm Ltd., "Arm Compute Library," 2021. [Online]. Available: https://github.com/ARM-software/ComputeLibrary

[16] Thundercomm, "RUBIK Pi 3," [Online]. Available: https://www.thundercomm.com/product/rubik-pi/. Accessed: 2026-07-08.

[17] Qualcomm Technologies, Inc., "Qualcomm® QCS6490/QCM6490 Processors Product Brief," Doc. No. 87-28733-1, Rev. F. [Online]. Available: https://docs.qualcomm.com/doc/87-28733-1/87-28733-1_REV_F_QUALCOMM_QCS6490_QCM6490_Processors_Product_Brief.pdf. Accessed: 2026-07-08.